

UNIVERSITY OF BIRMINGHAM

TESTING WINDOWS CODE INTEGRITY FROM SOFTWARE

by

MUHAMMED TARIQ ALI RAZVI

A dissertation submitted to the University of Birmingham for the degree of
MSc in Cyber Security

Supervised by Dr Tom Chothia
School of Computer Science
University of Birmingham
September 2026

© Copyright by MUHAMMED TARIQ ALI RAZVI, 2026
All Rights Reserved

Abstract

Windows tells its users that kernel mode code is safe because Microsoft has checked it. A driver is allowed to load because it carries a Microsoft signature, and that signature is presented as an assurance that someone competent has examined the driver and approved it. Very little of the machinery behind that claim is open to inspection. The components that enforce it are undocumented internal libraries, the vulnerable driver blocklist acts on a binary policy and not a published rule set, and an outsider has almost no way of telling what an approved driver is actually permitted to do once it is loaded. Obscurity is not a security property. It only makes the claim hard to check.

This dissertation checks one corner of that claim directly. Recent work at the University of Birmingham reached raw physical memory on Windows by inducing memory aliasing, and read that memory using WinPmem, a correctly signed forensic driver, through the user mode program that comes with the tool. This work returns to the same driver, addresses its device interface directly, and asks what it gives to an administrator on an ordinary machine with no aliasing induced at all. The machine was a Windows 10 guest under VirtualBox, which puts the newest layer of Windows defence, VBS and HVCI, outside what this work tested and leaves the signing model itself squarely inside it.

Reading is not protected. The driver handed over the whole physical address space of the machine, with no unsigned code, no exploited vulnerability and no aliasing, and it handed over the contents of processes that had already exited as well. Writing is a different matter. The device granted a write handle without complaint, so the barrier was never permission, and every attempt still brought the machine down before the value it was aimed at had changed. What stopped this attack was not the signing check that Windows advertises as the barrier, and it was not a permission check of any kind, but a check of a different sort, built into the kernel to notice when its own structures stop making sense, which halted the machine deliberately on three of the four runs that left a record. A routine content update to the target application delayed the work only until the new offsets were found, and that is not a security control either.

Acknowledgements

I would like to thank my supervisor, Dr Tom Chothia, for his guidance throughout this project, and in particular for the suggestion that a negative result which is properly measured is worth more than a positive result which is not. I am grateful to the Security and Privacy group at the University of Birmingham, whose recent work on memory aliasing provided both the starting point and the vocabulary for this investigation.

Declaration

I certify that this report is my own work. The tooling, the test environment, the experiments and the results reported here are mine. The overlay used at stage 2 is an existing open source project and is identified as such in Chapter 4.

I used AI assistants, Claude and Perplexity, during this project. They were used for help with specific coding problems, for generating figures, for finding and reviewing literature, and, working from my own notes, records and results, for drafting and editing the text of this report, which I then reviewed, corrected and finalised. All outputs have been checked against my own work and edited where they did not reflect it accurately, and I take full responsibility for the content of this report.

Contents

Abstract i

Acknowledgements ii

Declaration iii

Contents iv

List of Figures vi

List of Tables vii

List of Acronyms viii

1 Introduction 1

1.1 Motivation . . . 1
1.2 Research question . . . 2
1.3 Contributions . . . 2
1.4 Scope and ethics . . . 3
1.5 Structure of this report . . . 3

2 Background and Related Work 4

2.1 Windows code integrity and its trust model . . . 4
2.2 Virtualisation Based Security . . . 5
2.3 Physical memory as the real boundary . . . 6
2.4 Memory acquisition and how accurate it can be . . . 7
2.5 Game memory as a test target . . . 7
2.6 The gap . . . 8

3 Threat Model and Methodology 9

3.1 Attacker model . . . 9
3.2 The five stage design . . . 9
3.3 Finding the live copy among the duplicates . . . 10
3.4 Confirming that a write took effect . . . 11
3.5 What the design cannot show . . . 11

4 Implementation **12**
4.1 Test environment . . . 12
4.2 One search, two sources . . . 13
4.3 Reading process memory . . . 14
4.4 Reading physical memory . . . 14
4.5 Writing, and knowing whether it worked . . . 15
4.6 The target . . . 15

5 Results and Evaluation **16**
5.1 Summary of outcomes . . . 16
5.2 Virtual memory . . . 16
5.3 Reading physical memory . . . 17
5.3.1 Data left behind by processes that have exited . . . 18
5.4 Writing to physical memory . . . 19
5.5 What could account for the write failures . . . 20
5.6 How long a result stays valid . . . 22
5.6.1 What a patch actually costs . . . 22
5.7 The scanner contaminated its own results . . . 23

6 Discussion, Limitations and Ethics **24**
6.1 What the results mean . . . 24
6.2 What could be done about it . . . 24
6.3 Limitations . . . 25
6.4 What has been shown and what has been argued . . . 25
6.5 Ethics . . . 27

7 Conclusion **28**
7.1 Conclusions . . . 28
7.2 Future work . . . 29
7.3 Closing . . . 29

References **30**

A The physical memory write path **32**
A.1 Code submitted with this report . . . 33

List of Figures

2.1 Windows Virtual Trust Levels 5

3.1 The five stage escalation 10

4.1 The test environment from both sides 12

5.1 Reading player positions from process memory 17

5.2 The guest after a write to physical memory 19

List of Tables

3.1 The five stages 10

4.1 Test environment 12

5.1 Stage outcomes 16

6.1 Every claim in this report against the evidence behind it. Measured means it was observed on the machine in Table 4.1, argued means it follows from what was observed and from published documentation, and nothing in the second group is offered as a result. 26

List of Acronyms

BYOVD Bring Your Own Vulnerable Driver. 4, 8

DIMM Dual In-line Memory Module. 6

DSE Driver Signature Enforcement. 4

EDR Endpoint Detection and Response. 5

EEPROM Electrically Erasable Programmable Read-Only Memory. 6

ESP Extra Sensory Perception. 7

HVCI Hypervisor Enforced Code Integrity. 1, 4–6, 13, 25, 29

PPL Process Protection Light. 5, 6, 13, 14, 24

SMBus System Management Bus. 6

SPD Serial Presence Detect. 6

VBS Virtualisation Based Security. 1, 3, 5, 6, 13, 25, 29

VTL Virtual Trust Level. 5

Chapter 1

Introduction

1.1 Motivation

Modern Windows security is built on who wrote the code, not on what the code does. A kernel driver is allowed to load because it carries a signature from Microsoft (Microsoft, 2026b). That signature says Microsoft looked at the driver and approved it. It does not say the driver is safe, and it says nothing about what the driver can do once it is running.

Microsoft has added further layers of that one check. PatchGuard watches for changes to important kernel structures (Russinovich, Solomon, and Ionescu, 2017). Virtualisation Based Security (VBS) runs the normal Windows kernel inside a virtual machine and puts a Secure Kernel in a separate, more privileged one (Microsoft, 2026d). Hypervisor Enforced Code Integrity (HVCI) then moves the code integrity check into that Secure Kernel, where the normal kernel cannot reach it (Microsoft, 2026a). On supported hardware, VBS is on by default in Windows 11.

Microsoft says this stack is meant to stop an attacker who is already an administrator (Microsoft, 2026d). That is a strong claim. On Linux, root is the end of the security model. Windows says admin is not, and companies rely on that. An admin on a work laptop is not supposed to be able to terminate the antivirus, load a driver of their choosing, or read the memory of a protected process.

The claim is also hard to check from the outside. The parts that enforce it, `securekernel.exe` and `skci.dll`, have no public documentation. The list of banned drivers ships as a binary file, not a list anyone can read. So most of those relying on protection are accepting it without proper evidence.

That has started to change. BadRAM showed that a memory module can be made to lie about its own size, which turns the top half of memory into an alias for the bottom half (De Meulemeester et al., 2025). Download More RAM did the same thing on Windows using software alone, and used it to break VBS and HVCI (Collins, Chothia, et al., 2026). Both start by breaking isolation and then read memory. This project goes the other way round. It asks what an administrator can already read and write before anything is broken, using only software that Windows already trusts.

Counter-Strike 2 is used as the live target, because game memory is already a well studied

case in this area and it changes often enough to test how long a result stays true (Collins, Poulopoulos, et al., 2024).

1.2 Research question

The central question of this report is:

> *How much protection does "signed by Microsoft" actually give, when the attacker is an administrator using only signed, legitimate software?*

This breaks into three parts, and the method in Chapter 3 is built to answer each one separately.

SQ1, reading. Can an administrator read raw physical memory without unsigned code, without an exploit, and without any driver on Microsoft's banned list?

SQ2, writing. Does the same route allow writing, and if it does, is the result usable?

SQ3, lifespan. Once a target has been found in memory, how long does that knowledge stay correct?

1.3 Contributions

1. **A five stage test.** Each stage adds one capability over the last, starting with a dummy process in virtual memory and ending with raw physical memory on a live machine. Because only one thing changes at a time, a failure points at a specific capability and not at the experiment as a whole.

2. **One scanner, two sources.** `pmem_scanner` runs the same search code against process memory through `ReadProcessMemory` and against physical memory through the WinPmem device. Keeping the search identical means any difference in the result comes from the access route and not from the search.

3. **Reading physical memory is not protected.** A signed WinPmem driver that is not on the banned list gives an administrator the whole physical address space. No unsigned code, no exploit, no aliasing.

4. **Writing is permitted but was never made to work.** Windows let an administrator open physical memory for writing without any objection, so nothing in the signing model was stopping it. The writes still failed. Every attempt brought the machine down, and the value being aimed at never changed in any run. Section 5.5 sets out the three mechanisms

that could account for that, shows which of them the evidence favours, and names the experiment that would separate them. The authors of Download More RAM reached the same conclusion about the same driver by a different route, finding it gave them reads but no usable write, and building a ramdisk overlay instead (Collins, Chothia, et al., 2026).

5. **Targets move.** A game update changed the fixed offsets the tool relied on and it stopped working, while the values found by signature scan kept working. Finding the new offsets is ordinary work and not a research problem, so the update delayed the attack without stopping it.

6. **A stated boundary for the claim.** Section 6.4 maps every claim in this report onto the evidence behind it and the threats to that evidence, including the protections that sat outside what was tested, so that what has been shown, and what has only been argued, can be told apart without reading the whole report.

1.4 Scope and ethics

All the work in this report was done on machines and virtual machines belonging to the author. No third party data was involved, and no managed or production system was touched. The work follows the principles set out in the Menlo Report (Bailey et al., 2012). Where something here could be used to cause harm, the report describes what the capability is and which boundary it crosses, rather than providing a working tool. No attack code is released. Chapter 6 covers the ethics in full, including the lines this project chose not to cross.

The scope is narrow on purpose. This project does not reproduce the memory aliasing used in Download More RAM, does not attack VBS enclaves, and does not build or release a working cheat. It only looks at what already signed software is allowed to do.

1.5 Structure of this report

Chapter 2 covers Windows code integrity, the tools used to read physical memory, and the related work this project builds on. Chapter 3 sets out the attacker model and the five stage test. Chapter 4 describes `pmem_scanner` and the test machines. Chapter 5 reports what each stage did, including the stages that failed. Chapter 6 covers what the results mean, the limits of the work, and the ethics. Chapter 7 concludes.

Chapter 2

Background and Related Work

This chapter covers what is needed to follow the rest of the report: how Windows decides that kernel code is allowed to run, the layer that was added because that first check only protects the kernel from itself, the physical memory sitting under all of it, the forensics tools built to read that memory, and the game cheating work this project takes its target from. It ends by showing the gap between them.

2.1 Windows code integrity and its trust model

Windows decides whether kernel code can run by asking where it came from. Since Windows Vista on 64 bit hardware, Driver Signature Enforcement (DSE) has required every kernel driver to carry a signature that traces back to a Microsoft root before the loader will load it (Microsoft, 2026b). PatchGuard adds to this by checking periodically that important kernel structures have not been changed, and deliberately halting the machine if they have (Russinovich, Solomon, and Ionescu, 2017).

The point that matters for the whole of this report is that a signature says who made the code, not what the code does. It records that Microsoft accepted the file for signing. It says nothing about what that file can do once it is loaded, and nothing about whether those things are safe in the hands of whoever runs it.

The result is visible in Bring Your Own Vulnerable Driver (BYOVD) attacks, where an attacker who is already an administrator loads a properly signed but buggy driver just to get at the kernel access that the bug gives them. Monzani et al. (2026) studied how real malware uses this and showed it has gone from an uncommon technique to a routine step in an attack. Microsoft's answer is to maintain a blocklist of drivers, holding hashes of known bad drivers and of revoked signing certificates, checked every time a driver loads and required before HVCI will run (Microsoft, 2026a).

A blocklist can only ever look backwards, because a driver is added only after its abuse has been observed. That leaves two gaps. The first is the time between a driver shipping and being listed. The second is where this report sits: a signed driver whose dangerous ability is not a bug at all, but the job it was written to do, is unlikely to be listed, since there is no defect to fix.

DSE and PatchGuard also share the same weakness, which is why the next layer was built.

Both protect the kernel from inside the kernel, at the same privilege level, with no enforced boundary between the mechanism and its subject.

2.2 Virtualisation Based Security

Virtualisation Based Security (VBS) adds the boundary the previous section identified as missing. Hyper-V runs underneath the operating system and splits the machine into Virtual Trust Levels (VTLs). The normal Windows kernel and every normal user mode program sit in VTL 0. A second, more trusted level, VTL 1, holds the Secure Kernel and Isolated User Mode (Microsoft, 2026d). Code in VTL 1 can look at and control VTL 0, but not the other way round. Figure 2.1 shows the layout.

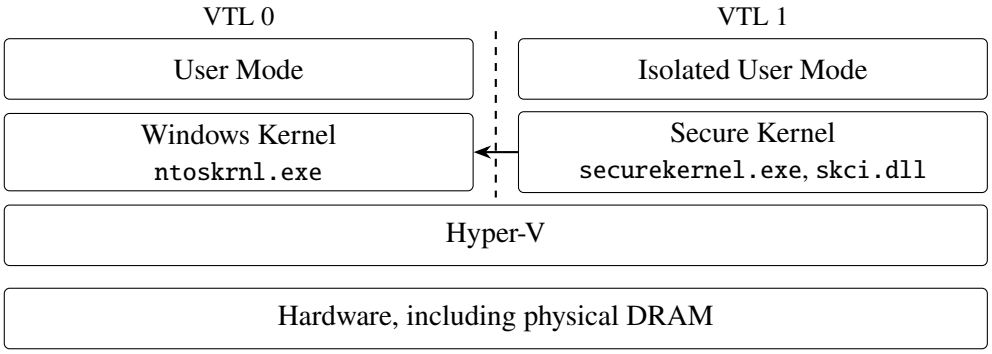


Figure 2.1: The Windows Virtual Trust Levels. The Secure Kernel in VTL 1 checks code loaded by the normal kernel in VTL 0, and that only works one way. Physical DRAM sits under every level and is shared by all of them.

Hypervisor Enforced Code Integrity (HVCI) moves the code integrity check into VTL 1, so the decision about whether a page of memory is allowed to run as code is made somewhere the normal kernel cannot touch. It also takes page table control away from that kernel, so a page can be writable or runnable but not both, and that rule is enforced from outside (Microsoft, 2026a). Process Protection Light (PPL) works in user mode instead. It gives Microsoft signed programs a protection level and stops anything at a lower level from reading, writing or killing them (Microsoft, 2026c). Antivirus and EDR services usually sit at level three, which is why an ordinary administrator cannot terminate them.

On supported hardware VBS is on by default in Windows 11, and Microsoft says the stack is meant to hold against an attacker who is already an administrator (Microsoft, 2026d). Most operating systems treat root as the end of the security model, and a lot of how Windows is deployed in companies depends on Windows not treating it that way.

Two things close this section. First, what PPL blocks is the handle route: `OpenProcess`, `ReadProcessMemory` and the rest. It says nothing about the physical memory that a protected process's pages are really sitting in. Second, none of this is easy to check from outside Microsoft.

The parts that enforce it, `securekernel.exe` and `skci.dll`, are undocumented. The blocklist ships as a binary file and not a list anyone can read. And the best public account of the Windows kernel (Russinovich, Solomon, and Ionescu, 2017) is older than the system it would need to cover. The claim is therefore difficult to check from outside, which is the gap this report works in.

2.3 Physical memory as the real boundary

Each of the mechanisms above operates through an intermediary. PPL controls handles, HVCI controls page permissions, VBS controls which level can see which. Underneath all of them is one block of physical DRAM that every level shares. Someone reading and writing physical addresses directly is not beating these mechanisms so much as working below the level they exist at.

Cold boot attacks showed this years ago, recovering encryption keys from memory modules removed from a running machine before their contents decayed (Halderman et al., 2009). BadRAM showed the same line can be crossed without taking anything out permanently. A DIMM reports its own layout from an EEPROM chip using the Serial Presence Detect (SPD) standard, and the memory controller accepts the reported values without validation. Change the recorded row address bits and the module claims twice the memory it really has. After that the top half of the address space is an alias for the lower half, and anything protected in one half can be reached through the other (De Meulemeester et al., 2025).

Download More RAM removed the need for physical access on Windows (Collins, Chothia, et al., 2026). The authors rewrote the SPD chip over the SMBus from software, which works on a large share of consumer memory because it ships without SPD write protection. They kept the machine stable with the `removememory` boot option, read the aliased memory with a modified build of WinPmem, got a limited write by putting a small FAT ramdisk over part of that region, and used it to patch the revocation checks in `skci.dll`. With the blocklist disabled they could load banned drivers and get clean access to all of physical memory. Microsoft issued CVE-2026-23670 and a partial fix.

Two details from that work matter most here. The first is why they picked WinPmem: its bounds checks live in the user mode program, not in the driver. The second is what they say about writing. On their account the driver does not “provide a direct write primitive” (Collins, Chothia, et al., 2026), which is the whole reason the ramdisk was needed. Chapter 5 returns to this point.

2.4 Memory acquisition and how accurate it can be

WinPmem was not built for attacking anything. It comes from digital forensics, where responders copy the memory of a running machine to recover keys, injected code and traces of processes that never touch the disk. Stüttgen and Cohen (2013) describe the design of an acquisition driver built to survive anti-forensic tricks, and the tool described there is the direct ancestor of the driver used in this project.

Forensics has been careful about what a correct copy of memory even means. Vömel and Freiling (2012) set out three tests, correctness, atomicity and integrity, and point out that a tool copying a live machine cannot freeze it while it works, so the image it produces is stitched together from moments that never existed at the same time. Pagani, Fedorov, and Balzarotti (2019) measured this, showing how far a copy drifts from any single instant and that the differences are big enough to change what an analyst concludes.

That literature is written from the defender's side, and it asks whether the copy is accurate. It does not ask the other question: a tool like this has to be able to read all of physical memory to do its job, and on Windows it has to ship a signed driver to do that, so the question of what it grants an administrator intending to misuse it has not been asked. The atomicity work also points at something on the write side, which Chapter 5 develops. If reading live memory without freezing it gives you an inconsistent picture, then writing to live memory without freezing it should be expected to give you an inconsistent machine.

2.5 Game memory as a test target

Games are useful for memory work because their state has to be fast, sits in memory in predictable shapes, and is subject to sustained attack. Yan and Randell (2005) give the classification the field still uses, where ESP overlays and wallhacks count as showing the player information they should not have. Alangari and Alharbi (2025) review current defences and confirm that cheats of this kind are still hard to stop from the server side.

Collins, Poulopoulos, et al. (2024) look at client side anti-cheat directly, with Counter-Strike 2 among their targets, and find that kernel level defences are not enough against a determined attacker on the machine. That work is why this project uses the same target, and it gives Chapter 5 something to compare against. Dorner and Klausner (2024) look at the same systems from a different angle, pointing out that anti-cheat drivers run in the kernel and behave in ways that are hard to tell apart from a rootkit. That is the same problem as Section 2.1, seen from the other side. Signing decides who is trusted, not what they are allowed to do once they are.

2.6 The gap

Three areas of work touch the question this report asks, and none of them meets the others. The aliasing work treats physical memory as an attack surface, but assumes either physical access or an alias that has already been created. The forensics work builds the tools that read physical memory, but only ever judges them on how accurate they are, never on what they give an attacker. The cheating work targets game state in memory, but assumes the attacker obtains access through BYOVD or a driver they wrote themselves.

None of them asks what a properly signed driver that is not on the banned list grants on an ordinary Windows machine, to an administrator who writes no unsigned code and exploits nothing. That is what the rest of this report answers.

This question has not been asked directly. The aliasing work begins by breaking isolation and then reads physical memory; this project inverts that order and measures what is available beforehand. It is the first study to isolate the signing model as the variable under test, granting the attacker administrative privilege and nothing else, and to report what that alone yields.

Chapter 3

Threat Model and Methodology

3.1 Attacker model

The attacker in this report is a local administrator on a Windows machine. That is the same starting point Microsoft says its defences are meant to survive (Microsoft, 2026d), and it is a realistic one. It covers a rogue employee, a support contractor, and any malware that has already escalated, which most malware does early.

What separates this attacker from the ones in the related work is what they are not allowed to do. They may not write or load unsigned code. They may not exploit a software vulnerability. They may not use any driver on Microsoft's banned list. They may not open the machine, modify the hardware, or reflash firmware. And they may not create memory aliases, so the route used by BadRAM (De Meulemeester et al., 2025) and Download More RAM (Collins, Chothia, et al., 2026) is excluded.

What remains is software Windows already signs and trusts. Constraining the attacker this far is the point of the design. Anything they still manage to do is something the signing model permits by design, not something that slipped through a bug. A bug can be patched. A permitted capability cannot, without changing what the model means.

The goal is to read and change the memory of a process the attacker does not own, including one that user mode protections would normally put out of reach.

3.2 The five stage design

The work is built as five stages, and each adds exactly one capability to the one before it. Stages 1, 3, 4 and 5 hold everything else still, the same matching code and the same 31 byte marker. Stage 2 is the exception, since it substitutes a live game and searches for a player name instead, which is what makes it the test of a target that does not cooperate. That matters, because when a stage fails it is then clear which capability failed, rather than the whole experiment being one result that either worked or did not. Figure 3.1 shows the progression and Table 3.1 sets out what each stage adds.

Stage 1 uses a purpose built target, `dummy_target.exe`, which puts a known 31 byte marker on the heap and does nothing else. Because the marker is known, any failure at this stage is a

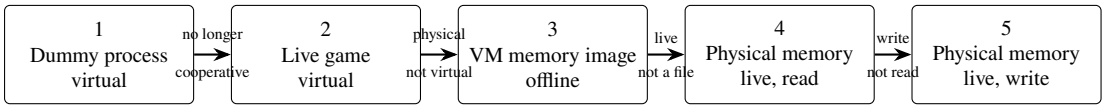


Figure 3.1: The five stages. Each arrow adds one capability and changes nothing else, so a failure can be attributed to the capability that was added, not to the experiment as a whole.

Stage	Source	Operation	What it adds
1	Dummy process, virtual	Read, write	Baseline, the target cooperates
2	Counter-Strike 2, virtual	Read, write	The target does not cooperate
3	VM memory image, offline	Read	Physical layout, no live system
4	`\\.\pmem`, live	Read	Live physical memory
5	`\\.\pmem`, live	Write	Changing it

Table 3.1: What each stage adds over the one before it.

failure of the tool and not of the target, which makes stage 1 the control for everything after it.

Stage 2 uses Counter-Strike 2. A game is a useful uncooperative target because their state has to be read and updated fast, it sits in memory in predictable shapes, and is defended (Collins, Poulopoulos, et al., 2024).

Stage 3 takes the same dummy target and the same marker off the live system. The virtual machine is paused and its state saved to a file on the host, and that file is then scanned as an ordinary file with the same matching code. What the stage tests is whether the search still works against a physical layout, where the ordered virtual view that a process presents is gone, before anything has to be read from a running machine.

Stage 4 is where the signing question is actually answered, because reading `\\.\pmem` needs a signed kernel driver and nothing else. Stage 5 asks whether the same route also allows change.

3.3 Finding the live copy among the duplicates

A scan of physical memory for a 31 byte marker does not return one result. It returns many, because the same bytes exist in several places at once: the heap copy the process actually reads, other copies the process made, buffers belonging to the console, the scanner's own copy of the search pattern, and pages that were freed long ago but never cleared. Only one of them matters, and a physical offset carries no label saying which process it belongs to.

The method used here finds it by difference, in three passes:

1. Scan physical memory with the target running.

2. Close the target, start it again, and scan a second time. Any offset in both scans cannot belong to the instance that was just created. Any offset only in the second scan appeared with it.

3. Scan a third time without restarting anything. This removes offsets that belonged to the previous scan's own memory instead of the target.

What survives all three filters is the live copy. Running the procedure is also what exposed a problem with the tool itself, reported in Section 5.7.

3.4 Confirming that a write took effect

The difficulty at stage 5 is not writing. It is knowing whether the write did anything. Three separate things can be true and they are easy to confuse:

1. `WriteFile` returns success. On its own this means nothing. It says the call was accepted, not that memory changed.

2. Reading the same physical offset straight back returns the new bytes. This shows the write reached physical memory.

3. The target process reports that its own value has changed. This shows the write reached the copy the process is actually reading, which is the only version of the claim that matters.

The tooling separates these. The scanner dumps the bytes at the target offset before writing, writes, re-reads the same offset and compares, then waits and re-reads again to see whether the value persisted or was put back. The target checks its own marker every two seconds and announces a change, so a successful write is reported by the victim, not assessed visually. This matters because the machine may not remain running long enough for anyone to notice.

3.5 What the design cannot show

Two things are worth noting before the results. The method proves presence and not absence: if a stage fails, that shows this approach did not achieve it on this machine, and not that it is impossible. And a physical offset does not identify its owner. The procedure in Section 3.3 gives good evidence about which offset belongs to the target, but it is inference from behaviour, not a translation from a virtual address. A driver that could translate virtual to physical directly would remove that uncertainty, and writing one was outside the attacker model set in Section 3.1.

The test machines and software versions are given in Chapter 4.

Chapter 4

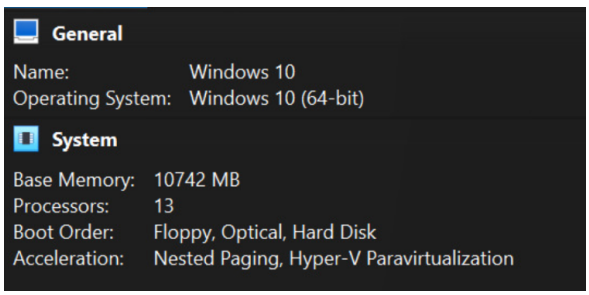
Implementation

4.1 Test environment

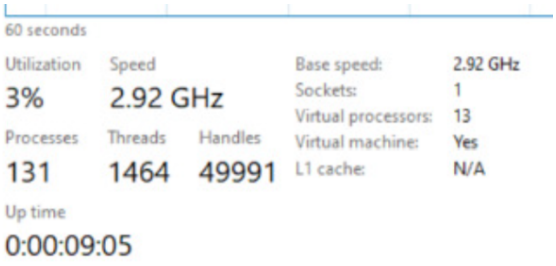
Every result in Chapter 5 was produced on one os, a Windows 10 virtual machine running under Oracle VirtualBox on a laptop with an Intel Core i7-13700H. Table 4.1 lists the configuration. Three details of it change how the results should be read, and all three are set out here so that nothing has to be inferred from the table later.

Component	Detail
Host CPU	Intel Core i7-13700H
Hypervisor	Oracle VirtualBox, nested paging enabled
Guest operating system	Windows 10 22H2, 64-bit, build 19045.3803
Guest memory	10 742 MB
Guest processors	13 virtual
Guest disk	50 GB VDI
Compiler	MSVC 19.51.36252, x64, C++17
Acquisition driver	WinPmem, loaded with `winpmem_mini_x64_rc2.exe -l`
Live target	Counter-Strike 2

Table 4.1: The machine and software used for every result in this report.



(a) What the guest was given, from the host.



(b) What Windows inside the guest reports.

Figure 4.1: The test environment seen from the host and from inside the guest. VirtualBox assigns 10 742 MB and 13 processors, and Windows inside the guest reports the same 13 and identifies the machine as virtual. The Hyper-V entry on the left is VirtualBox's paravirtualisation interface setting and not the hypervisor running the machine, which was Oracle VirtualBox throughout.

The first is that this is a guest and not bare metal. When the scanner opens the physical memory device it receives the physical address space the guest operating system has been given, which is the 10 742 MB in the table, and not the memory installed in the laptop underneath it. Everywhere this report says physical memory, it means the physical address space of the machine the code was running on, and that machine is the guest. Windows confirms the setting itself, since Task Manager reports the machine as virtual and lists 13 virtual processors.

The second is that the newer Windows protections were not active. VBS and HVCI are enabled by default on Windows 11 with supported hardware and are not enabled on a default Windows 10 install, and running them inside a VirtualBox guest needs nested virtualisation that has to be enabled deliberately. This was not checked directly while the tests were running, so the accurate statement is that the protections known to be active were driver signature enforcement, the vulnerable driver blocklist and PPL, and that VBS and HVCI sit outside what this work tested.

The third is the patch level. Build 19045.3803 corresponds to the cumulative update of December 2023, so the guest was approximately two and a half years out of date, but the test was also reproduced on a windows vm which was updated till early 2026. This is stated because it limits how far the results carry. The capability demonstrated in Chapter 5 is not a bug that a later update might have closed, since reading physical memory is what an acquisition driver is written to do, but that is an argument and not a measurement, and no fully patched system was tested here. Chapter 6 returns to both points.

4.2 One search, two sources

The tool written for this project is `pmem_scanner`, a byte pattern scanner in C++. Its design has one important property: the same matching code runs against both sources, and the only thing that differs is how the bytes are obtained. If the search itself changed between stages then a difference in the results could be a difference in the search, and nothing could be concluded about the access path. Holding the search still means any difference that shows up belongs to the path.

A pattern is given either as text or as raw hex, and the hex form matters more than it first appears. Passing the pattern as text leaves a copy of it in the scanner's own command line, in its heap, and in the console buffers, and all of those then appear as matches when the scanner searches the machine it is running on. Passing hex means the bytes being searched for never exist as those bytes anywhere in the scanner's arguments.

Matching is a plain byte comparison across a buffer, split over worker threads. Each thread is given a short lookahead past the end of its own segment so that a match lying across a thread boundary is still found, but it reports a match only if the match starts before that boundary.

Exactly one thread is then responsible for any given match, and no duplicates have to be removed afterwards.

The tool written for this project is `pmem_scanner`, a byte pattern scanner in C++.[1]

4.3 Reading process memory

The virtual path is the conventional one. `OpenProcess` takes a handle with `PROCESS_VM_READ`, `VirtualQueryEx` walks the address space to list the committed readable regions, and `ReadProcessMemory` pulls each region into a buffer to be searched. Region enumeration has to run in sequence because each query needs the end address of the previous region, so only the read and search work is spread across threads.

This is the path that user mode protections govern. It is also the path that PPL closes off and that client side defences watch.

4.4 Reading physical memory

The physical path uses none of those calls. With the WinPmem driver loaded, its device is opened as though it were an ordinary file:

```
HANDLE hDevice = CreateFileW(
    L"\\\\.\\pmem",
    GENERIC_READ | (doPatch ? GENERIC_WRITE : 0),
    FILE_SHARE_READ | FILE_SHARE_WRITE,
    nullptr, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, nullptr);
```

Listing 4.1: Opening physical memory

After that it is `SetFilePointerEx` to seek and `ReadFile` to read, the same two functions any program uses on any file. Memory is swept in 64 MB chunks with the last 4 KB of each chunk carried into the front of the next, so a pattern lying across a chunk boundary is not missed, and ranges that cannot be read at all, such as memory mapped I/O holes, are skipped so the sweep continues past them. Those ranges are not memory being withheld, they are addresses with no readable memory behind them, so a sweep that skips them still covers the whole physical address space and still reads everything in it that holds anything.

Nothing in that listing is unusual. There is no exploit here and no novel technique. The capability comes from the driver having been signed, and the interface to it is file I/O.

[1] https://git.cs.bham.ac.uk/projects-2025-26/mxr1022

4.5 Writing, and knowing whether it worked

Writing uses the same handle opened with `GENERIC_WRITE` and `WriteFile` in place of `ReadFile`. The replacement value has to be the same length as the search pattern, since anything longer would run into neighbouring memory.

The first version of the tool wrote to every match the scan returned, and that was a mistake. A scan returns many matches and most of them are not the target, so writing to all of them guarantees damage somewhere unintended and makes any crash impossible to attribute to a single write. The tool was rebuilt to write to one location only, addressed by absolute physical offset so that it cannot drift between runs, and it refuses to write at all if the pattern is no longer sitting at that offset when the write is attempted.

The write is surrounded by the checks that give the outcome its meaning. The bytes at the target offset are dumped before anything is written. The same offset is read straight back afterwards and compared against what was sent. After a delay it is read once more, which separates a value that stayed from one that was put back by something else. The whole of physical memory is then swept again for both the old pattern and the new one so the counts can be seen to move. Output is written unbuffered, because this experiment bugchecks the machine and a bugcheck discards anything still waiting in a buffer. The code for the write and the checks around it is reproduced in Appendix A.

4.6 The target

`dummy_target.exe` is a purpose built target. It puts a known 31 byte marker on the heap, touches it once a second so it stays resident, and otherwise idles.

Two decisions in it are deliberate. The marker is assembled at runtime from fragments and never printed, because a program that prints a string creates further copies of it in console buffers and every copy becomes another false match in a physical scan. The target also checks its own marker every two seconds and reports it if the value has changed, so a successful write is recorded by the target itself. This matters because the machine may not remain running long enough for anyone to read the screen.

Chapter 5

Results and Evaluation

5.1 Summary of outcomes

Table 5.1 gives the outcome of each stage. In summary, every stage involving reading succeeded and everything involving writing physical memory did not, and the reason writing did not has nothing to do with code signing.

Stage	What was attempted	Result	Section
1	Dummy process, virtual, read and write	Worked	5.2
2	Counter-Strike 2, virtual, read and write	Worked	5.2
3	VM memory image, offline, read	Worked	5.3
4	Live physical memory, read	Worked	5.3
5	Live physical memory, write	Froze or crashed the guest	5.4

Table 5.1: Outcome of each stage of the escalation.

5.2 Virtual memory

Stage 1 behaved as designed. The 31 byte marker planted by `dummy_target.exe` was found through `ReadProcessMemory` and overwritten, and the target reported the new value on its next tick. Since the target was built to be found, this stage proves only that the tool works, which is what a control is for.

Stage 2 removed the cooperation and pointed the same code at a live Counter-Strike 2 process, searching for a known marker in the form of a player name. It was found and overwritten. One detail from this stage affected the later stages. TThe search returned several matches rather than one, which is the duplication problem set out in Section 3.3, and only one of them was the copy the game was actually using.. The live copy was identified by patching each in turn and watching which one the game reflected back. At this stage that is a small inconvenience, since a single process address space is a manageable size and there is a process available to query. It became the central difficulty once the search moved to physical memory, where neither of those is true.

The same access path was then used for something more substantial than a marker. Working from a process handle, the tool located the view matrix and the entity list in `client.dll` by

scanning for byte signatures, walked the entity list to reach each player, and read the world position, health and team number of each one. Those values were passed to an external overlay, which converted world positions into screen coordinates using the view matrix and drew player positions through walls. The result worked.



Figure 5.1: Values read from the game's memory and drawn over it. On the left the tool locates the view matrix, the entity list and the player controller by signature scan at launch, then reports the offsets it found. On the right those values are drawn as boxes over bot players, including ones behind cover.

It is worth being precise about what this does and does not show, because it is easy to overstate. This was virtual memory access through a process handle, which is the conventional route and the one that user mode protections govern. The physical memory capability demonstrated in Section 5.3 was shown against the dummy target, and there was a single attempt to carry it through to the game. Closing that distance is the main piece of work this project tried to finish but could not, and Section 7.2 returns to it.

5.3 Reading physical memory

Stage 3 moved the search off a live process altogether. The virtual machine's state was saved and the resulting image was scanned as an ordinary file. The same matching code found the marker with nothing changed except the source the bytes came from, which confirmed that the search worked against a physical layout and not only against the ordered virtual view a process presents.

Stage 4 is where the research question is actually answered. With the WinPmem driver loaded, `\\.\pmem` was opened and 10 742 MB of physical memory was swept in 64 MB chunks across 13 search threads. The marker was found. No unsigned code was written, no vulnerability was exploited, no driver on Microsoft's banned list was used, and no memory aliasing was induced. The requirements were administrative privilege and a driver Microsoft had signed.

```
[*] Scanning ~10741 MB of physical memory
[*] Pattern: 31 byte(s), 13 search thread(s) per chunk
[+] Scan complete, 14 match(es).
```

```
[HIT   0] physical offset 0x105826cd1
[HIT   1] physical offset 0x11031e9e0
[HIT   2] physical offset 0x113a0c3e0
[HIT   3] physical offset 0x1828b5084
[HIT   4] physical offset 0x1828b50d5
[HIT   5] physical offset 0x18d19d27a
[HIT   6] physical offset 0x1a4daf5db
[HIT   7] physical offset 0x1a4daf661
[HIT   8] physical offset 0x1b6c5c4b9
[HIT   9] physical offset 0x1e32cb381
[HIT  10] physical offset 0x1e32cb3b9
[HIT  11] physical offset 0x21a0f6981
[HIT  12] physical offset 0x21a0f69b9
[HIT  13] physical offset 0x223c74db0
```

Listing 5.1: A physical memory scan for the 31 byte marker

This is the main finding, and nothing malfunctioned to produce it. A forensic acquisition driver that can read all of physical memory is a driver doing exactly the job it was written to do. There is no defect here to report and nothing for Microsoft to patch. The capability is what the signature was granted for.

Before the content update of 12 August 2026 the physical memory path was pointed at Counter-Strike 2 directly. The same wildcard pattern search used for the dummy target, with `??` standing for bytes that were allowed to vary, was run against `\\.\pmem` through WinPmem, the player values were located, their offsets were held in the tool, and an overlay drew from those offsets while the game ran offline against bots, with FACEIT's client side anti-cheat active. When the update landed, the working version was edited in an attempt to recover the new offsets and the original was lost in the process, so no scanner output, screenshot or build of it remains and it could not be repeated on the new build. Because nothing of that run was retained, it is withdrawn as a finding of this report and is recorded here only as an account of what was attempted.

5.3.1 Data left behind by processes that have exited

An unplanned result emerged from the procedure used to locate the live copy. Physical memory was scanned, the target process was closed and started again, and physical memory was scanned a second time. Twelve offsets appeared in both scans.

Those twelve were the previous instance's heap pages. The process had exited, its memory had been handed back to the system, and the marker was still present in full. Windows does not clear a page when it is freed, only when it is given to a new owner, so anything holding read access to physical memory can recover data belonging to processes that no longer exist. The read capability reaches backwards in time as well as across the running system, and nothing in the signing model distinguishes between the two.

5.4 Writing to physical memory

Stage 5 failed, and the way it failed is informative.

Nothing refused the request. The device was opened with `GENERIC_WRITE` and the request succeeded. Neither the driver, nor the code signing model, nor the banned driver list objected to an administrator asking for write access to the whole of physical memory. The barrier was not permission.

Two different sets of attempts were made. The first wrote to every match the scan returned, about five times, and every attempt crashed the machine. Since a scan returns many matches and most of them are not the target, those failures could not be the responsibility of any single write, so the experiment was rebuilt to write to exactly one offset, chosen by absolute address, and five or more attempts were made that way with the same outcome.

The manner of failure was the same in both designs. Most runs ended with the display filling with a repeating pattern of corrupted pixels while the guest stopped responding, as in Figure 5.2, and a minority ended in a bugcheck, the stop error Windows raises when the kernel finds it cannot safely continue.

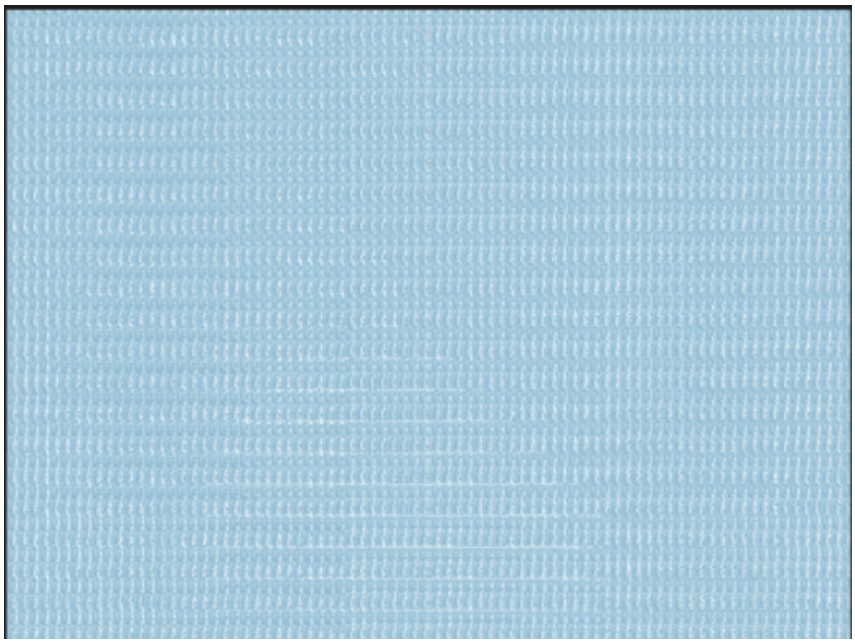


Figure 5.2: The guest immediately after a write to physical memory. The display has filled with a repeating pattern of corrupted pixels and the machine has stopped responding. This was the usual outcome, with a bugcheck appearing only occasionally.

That consistency is the useful result. If the failures came from hitting something the system depended on, a single write into the heap of an idle process should usually have been harmless, twenty scattered writes should have been far more likely to break something, and the two sets of runs should have looked different from each other. They did not. One write did the same damage

as twenty, which points away from what the bytes overwrite and toward the write operation itself. Mapping a frame of live memory writable through this interface appears to disturb the machine regardless of what is written into it. Something about how this interface commits a write to live physical memory disturbs the machine whatever it is aimed at.

This is not a null result. A write that changed nothing would not crash the kernel or corrupt the display, so the bytes issued through the device were reaching live memory that something else was reading.

What none of it shows is that the write landed at the address it was aimed at. The target was 31 bytes of a string on the heap of an idle process that only reads it, and overwriting that should have been harmless, since the process would have printed the new value and carried on, which is exactly what it was built to do. It never did. The write reached memory and the machine came down, but the target offset was unchanged.

That reading fits the surrounding literature in two directions. It matches Collins, Chothia, et al. (2026), who used the same driver for reading, reported that it gave them no direct write primitive, and built a ramdisk overlay for that reason. Two attempts, reached independently and by different routes, arrived at the same conclusion about the same driver. It also matches what the acquisition literature would predict. Vömel and Freiling (2012) and Pagani, Fedorov, and Balzarotti (2019) establish that a tool cannot freeze a live machine while it works, so a read produces an image assembled from moments that never coexisted. The same argument applied to writing says a write to live memory races the system it is writing to, and in these runs the write did not complete as intended.

5.5 What could account for the write failures

The runs in Section 5.4 establish two things and settle nothing beyond them. Bytes issued through the device reached live memory, since a write that changed nothing could not have corrupted the display or brought the kernel down, and the value at the target offset never changed in any run. Three mechanisms can produce that pair of observations, they carry different consequences for what a signed acquisition driver actually grants, and they are worth separating rather than collapsing into a single unexplained failure. Four kernel dumps survive from the runs that stopped rather than hung, and they narrow the field before the three are compared. Three report bugcheck `0x139`, `KERNEL_SECURITY_CHECK_FAILURE`, which Windows raises when a check built into the kernel finds one of its own structures in a state that structure is not allowed to be in, and which exists to halt the machine deliberately instead of letting it run on internal state that is no longer consistent. The fourth reports `0x0A`, `IRQL_NOT_LESS_OR_EQUAL`, recording a read of address `0xFFFFB90228D88008` at an interrupt level at which that address could not be touched, which is a different failure from the other three. What is absent matters

as much as what is present. No dump reports `ATTEMPTED_WRITE_TO_READONLY_MEMORY`, the code a write refused for want of write permission on the frame would carry, and none reports `CRITICAL_PROCESS_DIED`, the code that writes to a structure that is critical to system operation would carry. The parameters recorded alongside the three security check failures are zero, so the structure that failed its check cannot be named from the dump headers alone. The first is that the read path and the write path do not agree on where a physical offset points. Reading is served by mapping a frame and copying out of it, writing by mapping a frame and copying into it, and if the two operations resolve an offset differently then every write in this project landed somewhere other than where the scan had found the marker. That would account for an unchanged target, it would account for damage appearing somewhere the experiment was not looking, and it is the explanation offered in Section 5.4.

The second is that the offset was right for the read and wrong for the target. The procedure in Section 3.3 identifies the live copy by behaviour and not by translation, as Section 3.5 says plainly, so the offset written to was the offset that survived three filters and not an offset known to belong to the target process. A write into a frame that once held the marker, or that holds a copy the process never reads, changes nothing the process can report while still being a write into live memory.

The third is that mapping a frame of live memory writable through this interface is itself what destabilises the machine, whatever bytes are placed in it. This is the mechanism the evidence favours, and the crashes say so more directly than the run behaviour does. One write did the same damage as twenty, the two designs failed the same way, and the target was 31 bytes on the heap of an idle process whose corruption cannot plausibly account for a kernel that decides it cannot continue. Damage that does not scale with the number of writes, and does not depend on what those writes touched, points at the operation and not at its target, and the kernel's own account of why it stopped points the same way.

None of the three is excluded outright, and the second and third can still both hold at once. The experiment that separates them is small and it is set out in Section 7.2: write into a region of physical memory known to hold nothing the kernel depends on, read the same offset straight back, and see whether the bytes are there. If they are, the first mechanism is out and the failure was one of targeting. If they are not, the write path does not commit where the read path looks. And if the machine crashes regardless, a region that is not in use, the third mechanism is the answer and the write capability this interface appears to offer is not a capability at all.

Two things follow for the wider question. A driver that grants a write handle it cannot honour is still a driver that grants a write handle, and the signing model approved that interface without any statement about which of its operations do anything, which is the same silence this report documents on the read side. And the reason the write did not work, on all three readings, is a property of memory management and not of code integrity, which places the one barrier this

project actually met outside everything the signature was ever asked to decide.

5.6 How long a result stays valid

On 12 August 2026 a routine content update to Counter-Strike 2 moved the values the tool depended on. The reads stopped returning anything usable and the work at stage 2 ended there.

The manner of the failure matters, because the tool locates things two different ways at once and only one of them failed. Module level pointers such as the view matrix and the entity list are found by scanning for byte signatures, and they survived, since the instructions surrounding them did not change. Member fields such as the player origin and health are fixed numeric offsets into a class layout, and an update that moves a single member invalidates every offset below it. The signatures still resolved. The offsets did not.

That is the answer to SQ3. The only thing in this project that successfully stopped the attack was a content patch, released for unrelated reasons, that happened to move some structure members. Not code signing, not the banned driver list, and not any client side defence.

The same fragility applies in the other direction. A defence that looks for known byte patterns in memory decays on exactly the same schedule and for exactly the same reason, and has to be maintained just as often. Locating a value by its content survives changes. Locating it by a previously recorded offset does not.

5.6.1 What a patch actually costs

The question worth asking is what restoring it would have cost.

The tooling locates values two ways and only one of them needs a person. Seven module level pointers, among them the view matrix and the entity list, are found by signature scan at every launch, so a patch that moves them costs nothing at all and the tool recovers without intervention. The other thirty six values in the offset table are fixed numbers written into the source, and those are what an update breaks.

Thirty six offsets is a bounded piece of work and not a research problem. Source 2 carries its own class layout information at runtime, which is what makes the member offsets recoverable at all, and the same signature scanning already used for the seven pointers can be applied to the rest. The maintenance burden is measured in hours of one person's time per patch, spread across a community that shares the results, and the project that displays these values ships an updater and a version file precisely so that it can be kept current on that schedule.

That changes what the August update means. It did not stop the attack. It required a few hours of rework, which is a delay and not a defence. The answer to SQ3 is that nothing stopped this attack, and the one thing that slowed it down was not a security control.

5.7 The scanner contaminated its own results

One methodological result deserves recording, because it distorted the data before it was noticed.

Successive scans of the same unchanged system returned growing numbers of matches: 14, then 16, then 21, then 26. The scanner was the cause. It reads physical memory in 64 MB chunks into a heap buffer, and each chunk holds a copy of whatever it has just scanned, including the pattern being searched for. When the buffer was freed those bytes stayed in the page, so the next scan found residue from the previous scan and counted them as hits.

The signature was visible in the data once it was suspected. Across three runs a single unpaired match appeared at physical offsets `0x20b4fb750`, `0x14c825750` and `0x1556cc750`: a different page every time, but the same offset within the page, which is the scanner's own copy of the search pattern landing in a freshly allocated buffer. The correction was to zero each chunk before releasing it.

The observation is worth keeping for two reasons. It is the same phenomenon as Section 5.3.1 seen from the other side, and it shows that the tool altered the state it was measuring. Reading physical memory is not passive.

Chapter 6

Discussion, Limitations and Ethics

6.1 What the results mean

The central result of this project is that a Microsoft signature is reassurance and not security. It is not a judgement about safety. WinPmem is correctly signed, it is not on the blocklist, and it behaved correctly throughout, doing what it is meant to do, which is read the whole of physical memory, because that is the job an acquisition driver exists to do. No bug was exploited, there is no vulnerability to report, and there is nothing for Microsoft to patch. Every defence Microsoft has built around signing is built around the question of whether a driver should be allowed to run. None of them asks what it will be able to do once it is running, and the two questions have different answers.

The blocklist is the clearest example. It works by naming drivers whose flaws have already been seen and reported, so it is always looking backwards, and a driver has to malfunction before it can qualify, which means a driver whose dangerous ability is not a flaw at all but the job it was written to do will never appear on it, because there is nothing about it to fix. The signing model has no category for a driver that is working exactly as designed and is dangerous because of it.

Section 5.3.1 adds to this. Twelve copies of the marker outlived the process that made them, sitting in freed pages Windows had not cleared, so read access to physical memory reaches data belonging to programs that have already exited, and nothing in the stack tells a live process from a dead one at that layer. PPL guards the handle, and nothing guards the memory underneath it.

6.2 What could be done about it

Each of the obvious remedies carries a cost, and those costs are worth stating.

Blocking acquisition drivers outright would close the path and would break incident response, which is the discipline these tools were built for, because a responder who cannot image memory cannot recover keys, injected code, or anything else that never touches the disk. That is a real loss, traded for closing a route that already needs administrative privilege to use.

A narrower answer is to treat opening a physical memory device as an event worth logging. Nothing in this project was quiet: loading a driver and opening `\\.\pmem` are both visible acts, and any endpoint product that logged them would have seen every step of Chapter 5 as it

happened. Detection is not prevention and it does not stop the attack. It does change the cost of it, against a situation now where the whole sequence passes as ordinary.

The remanence finding has the cleanest answer of the three. Zeroing a page when it is freed, instead of when it is reallocated, would remove that whole class of disclosure, and it would cost performance on every free, which is presumably why it is not done. That is a deliberate trade off and not an oversight.

6.3 Limitations

All of this was done on one machine, and that machine was a VirtualBox guest running Windows 10 22H2 at build 19045.3803, around two and a half years behind current, and three things follow from it. The physical memory the scanner read was the guest's address space and not the laptop's installed memory, so the finding holds for the machine the code ran on and has not been shown on bare metal. VBS and HVCI were almost certainly not active and were never checked either way at the time, so nothing here touches the newest layer of Windows defence. And no current, fully patched system was tested at all. The argument that patching would not matter, because reading physical memory is what the driver is for, is a good argument, and it is still an argument.

The write results carry the heaviest caveat of anything in this report. Every attempt crashed the machine, which shows bytes reached live memory, and no attempt ever produced a confirmed change at the address it was aimed at. The explanation offered in Section 5.4, that the read path and the write path may not agree on where an offset points, fits the evidence and fits what Collins, Chothia, et al. (2026) report, and it has not been measured directly. Roughly five attempts of each kind is a small sample.

The largest limitation is one of reach. The physical memory capability was shown against a target built to be found, and the single attempt to carry it through to the live application was not repeated and left nothing behind, so the work that can be checked stayed on the conventional virtual memory path. That attempt is withdrawn as a result for the reason given in Section 6.4, and what survives of it is a direction and not a finding.

6.4 What has been shown and what has been argued

The claims in this report do not all rest on the same weight of evidence, and the difference matters more here than it usually would, because the central finding is a capability that worked and the second is a capability that did not. Table 6.1 separates them.

Table 6.1: Every claim in this report against the evidence behind it. Measured means it was observed on the machine in Table 4.1, argued means it follows from what was observed and from published documentation, and nothing in the second group is offered as a result.

Claim	Basis	Principal threat
An administrator can read the whole physical address space through a signed, non-blocklisted driver	Measured, Section 5.3	External. One VirtualBox guest, Windows 10 22H2 build 19045.3803, with VBS and HVCI outside the configuration
Freed pages retain the data of processes that have exited	Measured, Section 5.3.1	Internal. Ownership of a physical offset is inferred from behaviour and not translated
The device grants an administrator a write handle	Measured, Section 5.4	None material
Bytes issued through the device reached live memory	Measured, four kernel dumps, Section 5.5	Statistical. Four dumps from roughly ten runs, since the runs that hung left no record
No write changed the value it was aimed at	Measured, Section 5.4	Statistical. Roughly ten runs
Why the writes failed	Argued from the crash codes, Section 5.5	Two of the three mechanisms are weakened and none is excluded outright; the experiment that separates them was not run
The read result would survive VBS and HVCI	Argued, this section	Not measured anywhere in this report
Fixed offsets decay on a content update where signatures do not	Measured once, Section 5.6	External. A single update to a single application

Three of those threats deserve more than a line in a table.

The first is the largest, and it is the one named in the title. This work tested what a signed driver grants an administrator, and it did not test Hypervisor Enforced Code Integrity, which sat outside the configuration throughout. There is a reason to expect the read result to survive that gap, and it is worth setting out as an argument and not as a finding. HVCI decides which pages of memory may be executed as code and moves that decision into a privilege level the normal

kernel cannot reach (Microsoft, 2026a). It does not govern whether an administrator may open a device object and read frames of physical memory through it, because no code is loaded and no page is marked executable at any point in Chapter 5. On that reading a current machine with VBS and HVCI enabled returns the same physical address space and the finding holds. The argument follows from what Microsoft documents these components to do, it is the reason this report claims the patch level does not matter.

The second runs through everything the physical path claims about ownership. A physical offset carries no label saying which process it belongs to, the procedure in Section 3.3 infers ownership from behaviour across three scans, and that inference is good evidence and not a translation. Every statement in this report about reaching a particular process through physical memory inherits that uncertainty, and the write results in Section 5.5 inherit it twice over.

The third is the one this project created for itself. The single attempt to carry the physical path through to Counter-Strike 2 left no output, no build and no screenshot, so it cannot be checked by anyone including its author, and it is withdrawn as a result for that reason. It is carried in Section 5.3 as a record of what was attempted and nothing in the conclusions rests on it. A result that cannot be produced on demand is not a result, and saying so is cheaper than defending it.

6.5 Ethics

All work was carried out on machines belonging to the author, inside a virtual machine, with no third party data present and no managed or production system touched at any point. The project follows the principles set out in the Menlo Report (Bailey et al., 2012).

Testing against a game raises a question that deserves a direct answer. All of it was done offline against bot matches, no other players were present at any point, and no one outside this project was affected by it. This was a deliberate choice. The technique reads the game's own memory and behaves the same way whoever is on the other side, so testing against real people would have added nothing to the result while touching players who never agreed to take part.

No attack code is released with this report, the overlay used at stage 2 is not named, and what is described here are capabilities and the boundaries they cross and not anything ready to run. There is also nothing to disclose in the usual sense. A vulnerability report needs a defect, and what Chapter 5 documents is the intended behaviour of a correctly signed driver doing its job. There is no party to report it to, and that is itself the finding.

Chapter 7

Conclusion

7.1 Conclusions

This report asked how much protection the phrase “signed by Microsoft” really gives, when the attacker is an administrator confined to signed and legitimate software software, and the answer splits cleanly between reading and writing.

On reading there is no meaningful protection. A correctly signed, non-blocklisted forensic driver handed an administrator the whole physical address space of the machine, all 10 742 MB of it, with no unsigned code, no exploited vulnerability and no memory aliasing, and it handed over the contents of processes that had already exited as well, because freed pages sit untouched until somebody new is given them. That answers SQ1, and it answers it without anything having gone wrong anywhere: the driver worked correctly from start to finish, and reading the whole of physical memory is the job it was signed to do.

On writing the route was open but the result was not usable. The device gave up a write handle without complaint, every attempt crashed the machine, about five writing to all matches and five or more writing to one chosen offset, and not one of them produced a confirmed change at the address it was aimed at. The crashes show bytes reached live memory, since a write that changes nothing cannot bring down a kernel, and they do not show the write landed where it was sent. That answers SQ2 as a qualified no. Section 5.5 sets out the three mechanisms that could account for it and weighs them against the four kernel dumps the runs left behind, three of which record a kernel security check firing on structures the kernel had found in a state they were not allowed to be in. The barrier was never permission, and it was never code integrity either. It was a check on the kernel’s own internal consistency, which is not advertised as a defence, doing the only defending that happened here.

On how long a result stays valid, the answer is until the next content update. The offsets the stage 2 tooling leaned on were invalidated by a routine Counter-Strike 2 patch on 12 August 2026, and the work stopped there. That answers SQ3, and it is worth saying plainly what stopped it. Not driver signature enforcement, not the vulnerable driver blocklist, and not any client side defence. A content patch did it, for reasons that had nothing to do with security.

7.2 Future work

Four pieces of work follow directly from where this one stopped.

The first is to carry the physical memory path through to a live application and record it properly. The capability was shown against a target built to be found, the single attempt on Counter-Strike 2 left nothing behind once the August update broke it, and a repeat that is captured while it runs would settle the one claim this report has had to withdraw. It would also remove the inference from differencing procedure in Section 3.3, since a target that can be instrumented can confirm which physical offset it is really reading.

The second is to test the read and write path asymmetry directly. Writing into a region of physical memory known to be unused, then reading it straight back before anything the kernel depends on can be disturbed, would settle whether the two paths agree on where an offset points. It is a small experiment and it closes the biggest open question in Chapter 5.

The third is to run the read on bare metal, on a current and fully patched Windows 11 installation with VBS and HVCI enabled. The argument that patching should not matter is a reasonable one and nobody has measured it.

The fourth is to find values by what they look like instead of where they sat. The tooling at stage 2 located module level pointers by byte signature and those survived the August update intact while the fixed member offsets did not, so a tool built entirely on pattern matching should outlive content updates that break one built on offsets. The same question runs the other way, at any defence that scans memory for known patterns.

7.3 Closing

Windows decides what may run inside its kernel by asking who wrote it. The results here suggest the more useful question is what that code will be able to do once it is running, and it is a question the signing model was never built to answer. The parts that enforce these decisions are undocumented, the list of banned drivers ships as a binary file nobody outside Microsoft can read, and most of the people leaning on this protection are taking it on trust. Obscurity is not a security property. It only makes the claim hard to check, and this report checked one small corner of it.

References

Alangari, Adwa and Ohoud Alharbi (2025). *A Systematic Review of Technical Defenses Against Software-Based Cheating in Online Multiplayer Games*. arXiv: 2512.21377.

Bailey, Michael et al. (2012). "The Menlo Report". In: *IEEE Security & Privacy* 10.2.

Collins, Sam, Tom Chothia, et al. (2026). "Download More RAM: Dismantling Windows Operating System Defences with Mischievous Memory". In: *USENIX Security Symposium*.

Collins, Sam, Alex Poulopoulos, et al. (2024). "Anti-Cheat: Attacks and the Effectiveness of Client-Side Defences". In: *Proceedings of the 2024 Workshop on Research on Offensive and Defensive Techniques in the Context of Man At The End (MATE) Attacks*. DOI: 10.1145/3689934.3690816.

De Meulemeester, Jesse et al. (2025). "BadRAM: Practical Memory Aliasing Attacks on Trusted Execution Environments". In: *IEEE Symposium on Security and Privacy (S&P)*.

Dorner, Christoph and Lukas Daniel Klausner (2024). "If It Looks Like a Rootkit and Deceives Like a Rootkit: A Critical Examination of Kernel-Level Anti-Cheat Systems". In: *Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES)*. DOI: 10.1145/3664476.3670433.

Halderman, J. Alex et al. (2009). "Lest We Remember: Cold-Boot Attacks on Encryption Keys". In: *Communications of the ACM* 52.5.

Microsoft (2026a). *Enable virtualization-based protection of code integrity*. URL: https://learn.microsoft.com/en-us/windows/security/hardware-security/enable-virtualization-based-protection-of-code-integrity (visited on 08/28/2026).

— (2026b). *Kernel-Mode Code Signing Requirements*. URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/install/kernel-mode-code-signing-requirements--windows-vista-and-later- (visited on 08/28/2026).

— (2026c). *Protecting Anti-Malware Services*. URL: https://learn.microsoft.com/en-us/windows/win32/services/protecting-anti-malware-services- (visited on 08/28/2026).

— (2026d). *Virtualization-based Security (VBS)*. URL: https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-vbs (visited on 08/28/2026).

Monzani, Andrea et al. (2026). "Unveiling BYOVD Threats: Malware's Use and Abuse of Kernel Drivers". In: *Network and Distributed System Security Symposium (NDSS)*.

Pagani, Fabio, Oleksii Fedorov, and Davide Balzarotti (2019). "Introducing the Temporal Dimension to Memory Forensics". In: *ACM Transactions on Privacy and Security* 22.2.

Russinovich, Mark E., David A. Solomon, and Alex Ionescu (2017). *Windows Internals, Part 1*. 7th ed. Microsoft Press.

Stüttgen, Johannes and Michael Cohen (2013). "Anti-forensic resilient memory acquisition". In: *Digital Investigation* 10. Proceedings of DFRWS USA 2013.

Vömel, Stefan and Felix C. Freiling (2012). "Correctness, atomicity, and integrity: Defining criteria for forensically-sound memory acquisition". In: *Digital Investigation* 9.2.

Yan, Jeff and Brian Randell (2005). "A Systematic Classification of Cheating in Online Games". In: *Proceedings of the 4th ACM SIGCOMM Workshop on Network and System Support for Games (NetGames)*. DOI: 10.1145/1103599.1103606.

Appendix A

The physical memory write path

This appendix reproduces the part of `pmem_scanner.cpp` that the central claim of Chapter 5 rests on: opening the physical memory device for writing, issuing a write to a chosen physical offset, and checking whether it landed. The rest of the source is submitted alongside this report.

Three things are worth noticing in it. The write handle is asked for through an ordinary `CreateFileW` call and the driver does not refuse it. The write itself is a seek and a `WriteFile`, the same two calls any program uses on any file. And the check afterwards exists because `WriteFile` returning success proves nothing on its own, since it reports that the call was accepted and says nothing about whether memory changed.

```
// Open the WinPmem device. GENERIC_WRITE is requested only when a patch was
// asked for. The driver does not refuse it.
HANDLE hDevice = CreateFileW(
    L"\\\\.\\pmem",
    GENERIC_READ | (doPatch ? GENERIC_WRITE : 0),
    FILE_SHARE_READ | FILE_SHARE_WRITE,
    nullptr, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, nullptr);

// Seek to an absolute physical offset and write there.
static bool WriteAt(HANDLE hDevice, uint64_t offset,
                    const std::vector<uint8_t>& data)
{
    LARGE_INTEGER li;
    li.QuadPart = static_cast<LONGLONG>(offset);
    if (!SetFilePointerEx(hDevice, li, nullptr, FILE_BEGIN)) return false;

    DWORD written = 0;
    if (!WriteFile(hDevice, data.data(),
                   static_cast<DWORD>(data.size()), &written, nullptr))
        return false;

    return written == data.size();
}

// The check that gives the write its meaning. Reading the same offset
```

```
    straight
// back and finding the new bytes is what shows they reached physical
    memory.
if (!WriteAt(hDevice, off, patchBytes)) {
    std::cerr << "[!] WRITE FAILED at 0x" << std::hex << off << "\n";
} else {
    std::vector<uint8_t> check;
    if (ReadAt(hDevice, off, check, patchBytes.size()) && check ==
        patchBytes) {
        std::cout << "[+] VERIFIED: read-back matches the patch bytes.\n"
            ;
    } else {
        std::cout << "[-] NOT VERIFIED: read-back differs.\n";
    }
}
```

Listing A.1: Opening the device, writing, and verifying

A.1 Code submitted with this report

pmem_scanner.cpp The scanner described in Chapter 4. One matching core over two sources, process virtual memory through `ReadProcessMemory` and physical memory through the WinPmem device, with single offset patching and read-back verification.

dummy_target.cpp The test target. Places a known 31 byte marker on the heap, never prints it, and reports if its own value changes.

mem_scanner.cpp A pattern scanner and pointer chain resolver for process virtual memory, used at stage 2.

WinPmem is third party and was used unmodified.

All of the code written for this project is available at https://git.cs.bham.ac.uk/projects-2025-26/mxr1022.